

**ENS 491/92
Presentation**

Implementing Deep Neural Networks (DNNs) Using GPU's

Group 219

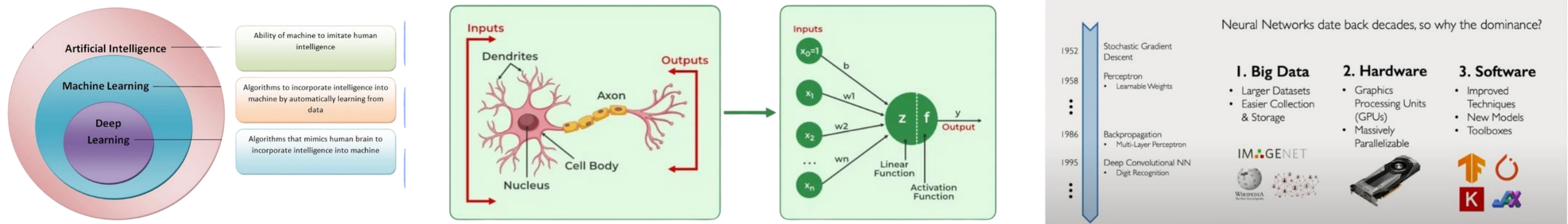
Ada Boran Yılmaz (30789)

Enes Çağlar (31109)

ENS 491 Design

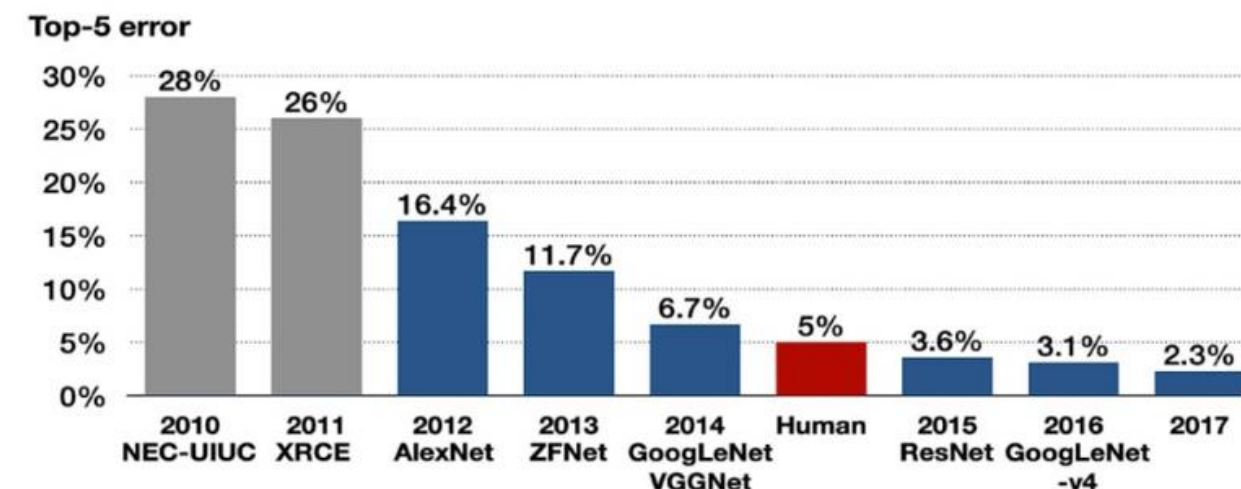
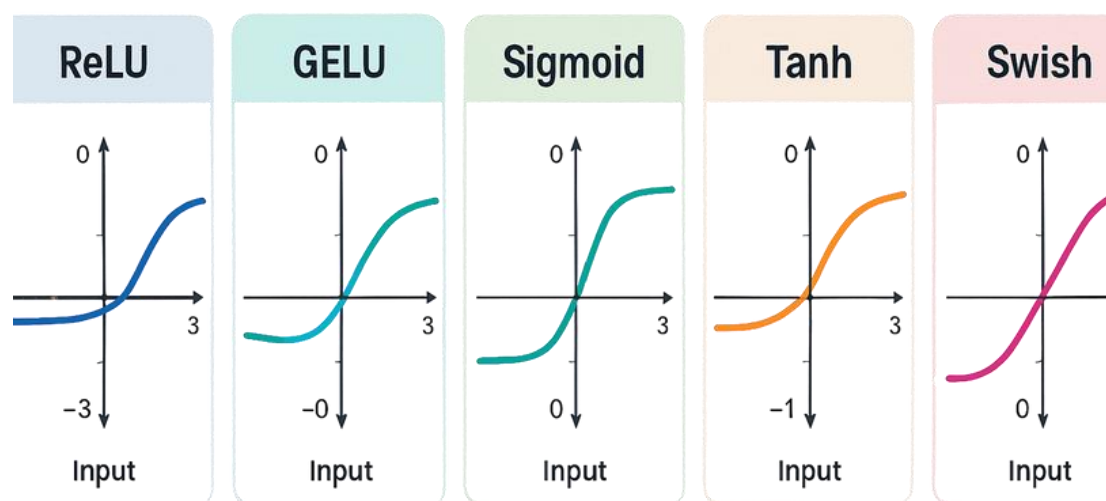
Theoretical Research & Deep Learning Fundamentals

- **Deep Learning Paradigms:** Explored the distinctions between Artificial Intelligence, Machine Learning, and Deep Learning algorithms designed to mimic human brain functions.
- **Historical Evolution:** Analyzed the rise of Deep Learning since 1952, driven by the emergence of Big Data (ImageNet) and parallelizable GPU hardware.



- **Core Components:** Researched neural network essentials including Activation Functions (Sigmoid, Tanh, ReLU), Backpropagation, and Gradient Descent optimization.
- **Benchmarking Datasets:** Evaluated performance metrics using standard datasets such as ImageNet (14M images) and Tiny ImageNet (100,000 images).
- **Overfitting Strategies:** Investigated techniques to improve model generalization, such as Dropout, Data Augmentation, and Early Stopping.

ACTIVATION FUNCTIONS IN NEURAL NETWORKS

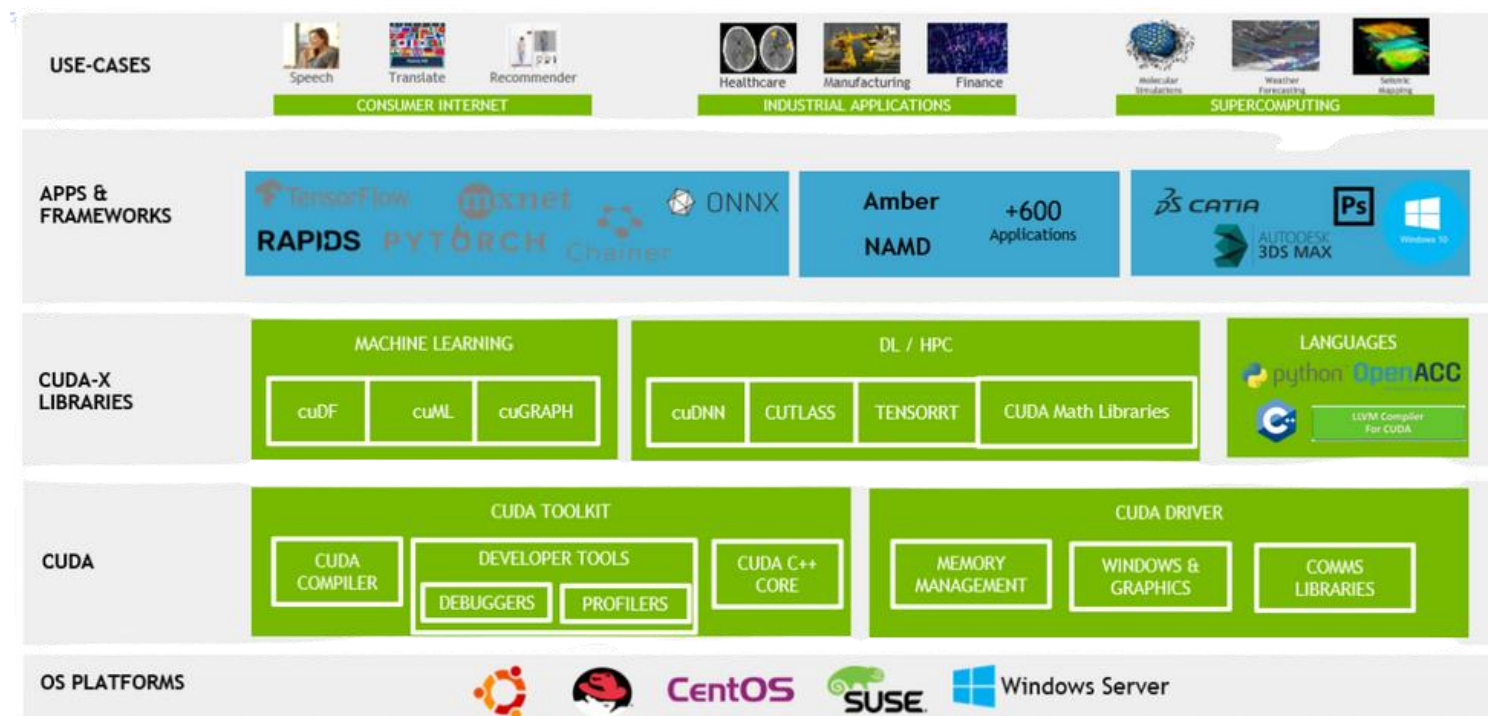


Tackling Overfitting

1. Dropout
2. Augmentation
3. Early Stopping

Technical Research & Hardware Optimization

- **CUDA Architecture:** Investigated GPU-accelerated computing using Compute Unified Device Architecture (CUDA) to facilitate massively parallel workloads.
- **Heterogeneous Computing:** Researched models to balance low-latency CPU processing with the high throughput capabilities of GPUs.



```
// CPU vector addition
void vector_add_cpu(float *a, float *b, float *c, int n) {
    for (int i = 0; i < n; i++) {
        c[i] = a[i] + b[i];
    }
}

// CUDA kernel for 1D vector addition
__global__ void vector_add_gpu_1d(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // one add, one multiply, one store
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

// CUDA kernel for 3D vector addition
__global__ void vector_add_gpu_3d(float *a, float *b, float *c, int nx, int ny, int nz) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int k = blockIdx.z * blockDim.z + threadIdx.z;
    // 3 adds, 3 multiplies, 3 stores
    if (i < nx && j < ny && k < nz) {
        int idx = i + j * nx + k * nx * ny;
        if (idx < nx * ny * nz) {
            c[idx] = a[idx] + b[idx];
        }
    }
}

#define N 10000000 // Vector size = 10 million
#define BLOCK_SIZE_1D 1024
#define BLOCK_SIZE_3D_X 16
#define BLOCK_SIZE_3D_Y 8
#define BLOCK_SIZE_3D_Z 8
// 16 * 16 * 8 = 2048
```

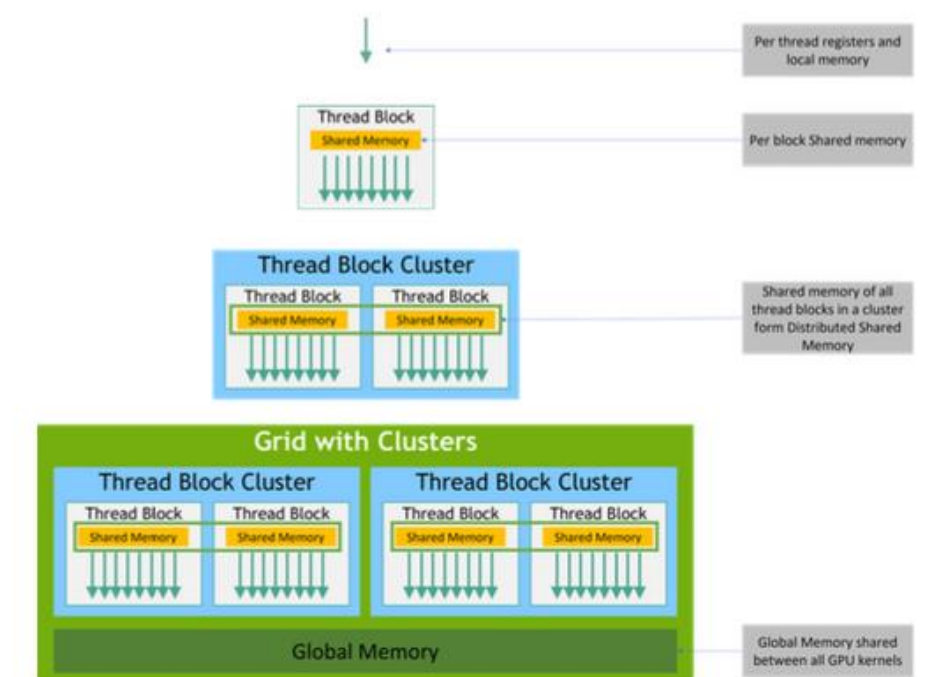
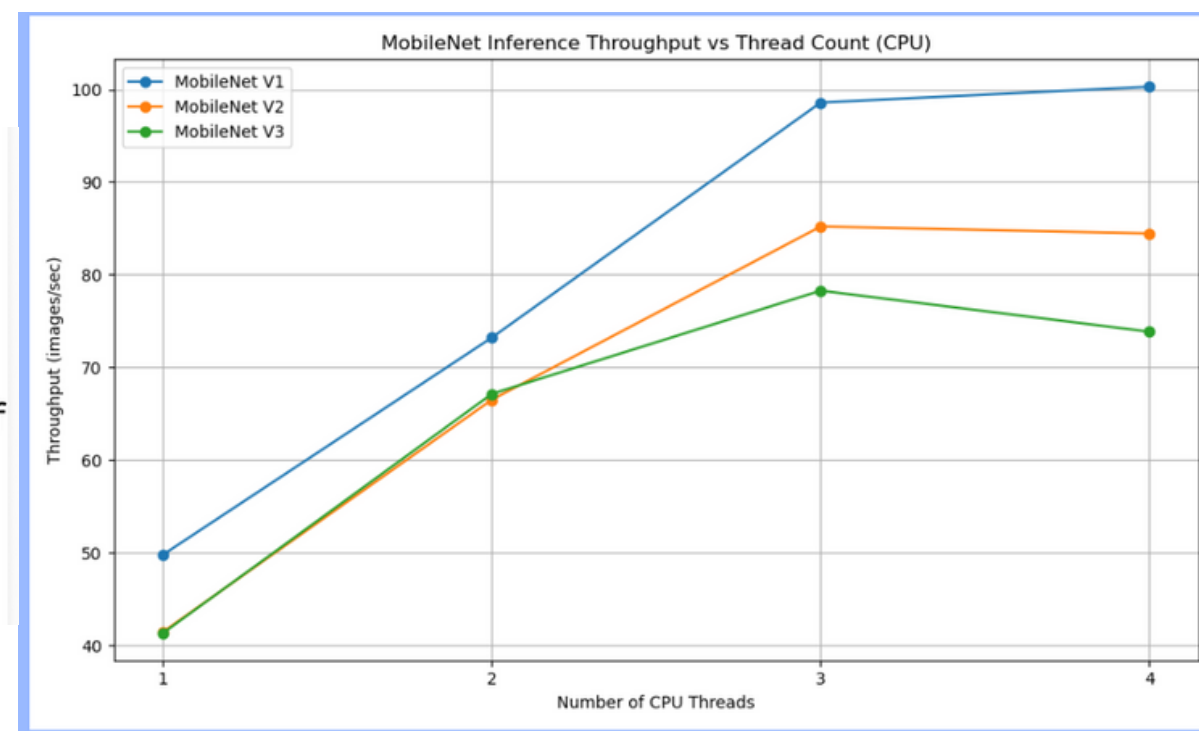
```
bash-5.1$ cat vector_add_v2.out
Performing warm-up runs...
Benchmarking CPU implementation...
Benchmarking GPU 1D implementation...
1D Results are correct
Benchmarking GPU 3D implementation...
3D Results are correct
CPU average time: 30.867660 milliseconds
GPU 1D average time: 0.198063 milliseconds
GPU 3D average time: 0.211172 milliseconds
Speedup (CPU vs GPU 1D): 155.847487x
Speedup (CPU vs GPU 3D): 146.172912x
Speedup (GPU 1D vs GPU 3D): 0.937923x
```

- **Lightweight Architectures:** Analyzed the MobileNet family (V1, V2, and V3) and EfficientNet-B0, focusing on depthwise separable convolutions to reduce parameters for edge device efficiency.
- **Performance Benchmarking:** Conducted initial CPU-based inference tests comparing throughput (images/sec) across different multithreading configurations.
- **GPU Memory Management:** Studied CUDA memory hierarchy, including global, shared, and register memory to optimize data transfer and kernel execution.

MobileNet v2 — logical continuation of the first model, developed in 2018.
It is 0.3x faster and smaller, +1% more accurate than MobileNet v1.

MobileNet v3 is the next generation of the MobileNet family, which is full of improvements to the MobileNet v2, developed in 2019.

It is 2x faster, 30% smaller, -3% accurate than MobileNet v2.



ENS 492 Implementation: MobileNet

High-Performance Computing on TRUBA

- **Infrastructure:** Utilized the Turkish National Science e-Infrastructure (TRUBA) for large-scale training and inference.
- **HPC Environments:** Configured workloads across different node types (Arf, Orfoz, Hamsi) with varying GPU and memory configurations.
- **Job Management:** Implemented sbatch scripts for automated scheduling and resource allocation on SLURM clusters.
- **Storage Optimization:** Leveraged the WEKA high-performance file system (/arf) to minimize I/O bottlenecks during data-intensive tasks.
- **Resource Scaling:** Tested performance across multi-CPU and GPU configurations to determine the optimal compute balance.



MobileNet Architecture & Efficiency Analysis

- **Computational Factorization:** Analyzed the shift from standard convolutions to Depthwise Separable Convolutions to reduce Mult-Adds.
- **Mathematical Modeling:** Calculated the theoretical speedup by comparing the costs of standard vs. separable operations.
- **Standard Benchmarking:** Evaluated MobileNet against popular architectures like GoogleNet and VGG16 in terms of parameter counts and ImageNet accuracy.
- **Resolution Multipliers:** Investigated the impact of varying input resolutions (e.g., 224 to 128) on total GFLOPs.

$$\text{Computational Gain} = \frac{\text{Computational Cost of MobileNet}}{\text{Computational Cost of Standard Conv}}$$

$$\text{Computational Gain} = \frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} = \frac{1}{N} + \frac{1}{D_K^2}$$

Table 6. MobileNet Width Multiplier

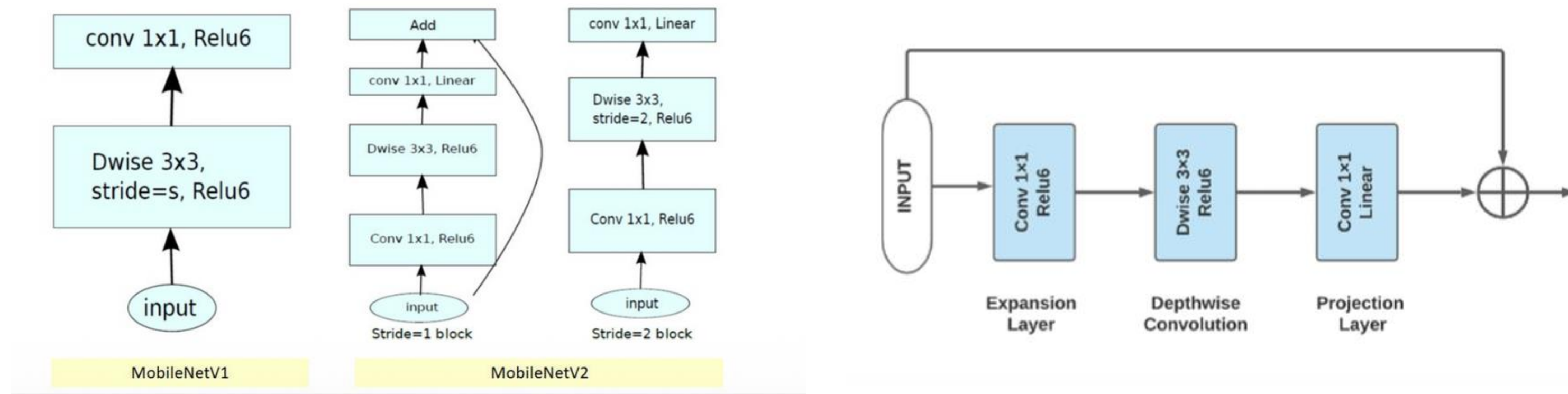
Width Multiplier	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
0.75 MobileNet-224	68.4%	325	2.6
0.5 MobileNet-224	63.7%	149	1.3
0.25 MobileNet-224	50.6%	41	0.5

Table 7. MobileNet Resolution

Resolution	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
1.0 MobileNet-192	69.1%	418	4.2
1.0 MobileNet-160	67.2%	290	4.2
1.0 MobileNet-128	64.4%	186	4.2

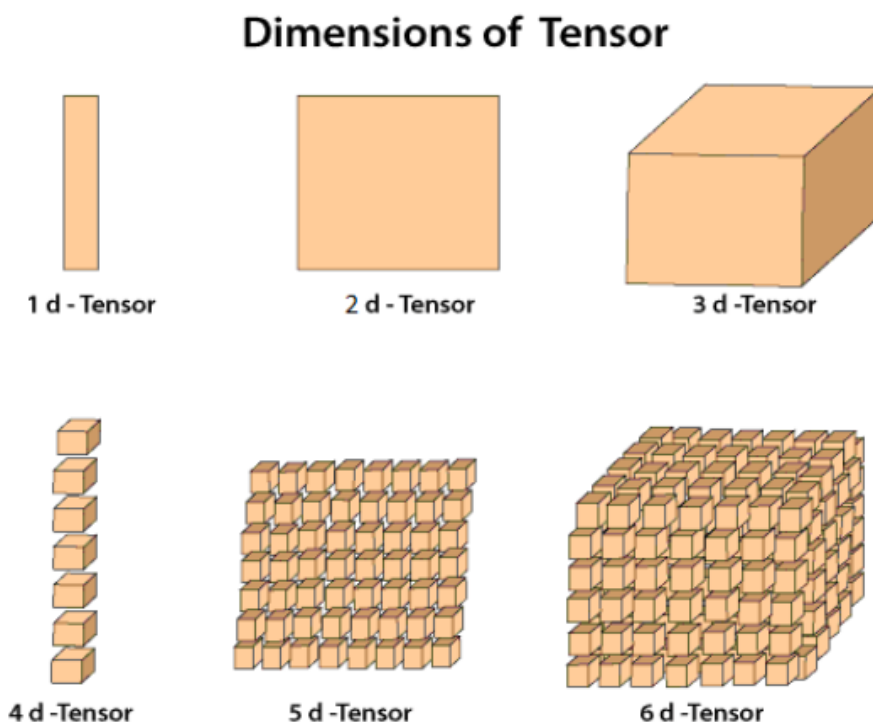
MobileNetV2: Inverted Residuals Implementation

- **Bottleneck Architecture:** Implemented the "Expand-Conv-Squeeze" structure with Linear Bottlenecks to preserve information in low dimensions.
- **Shortcut Connections:** Integrated residual links between narrow layers to facilitate gradient flow and improve training stability.
- **Deployment Mapping:** Mapped the architectural structure to custom tensor formats for later migration to low-level languages.
- **Inference Logic:** Defined the forward pass logic, ensuring proper handling of stride-2 blocks for spatial downsampling.



C++ Forward Pipeline & Tensor Design

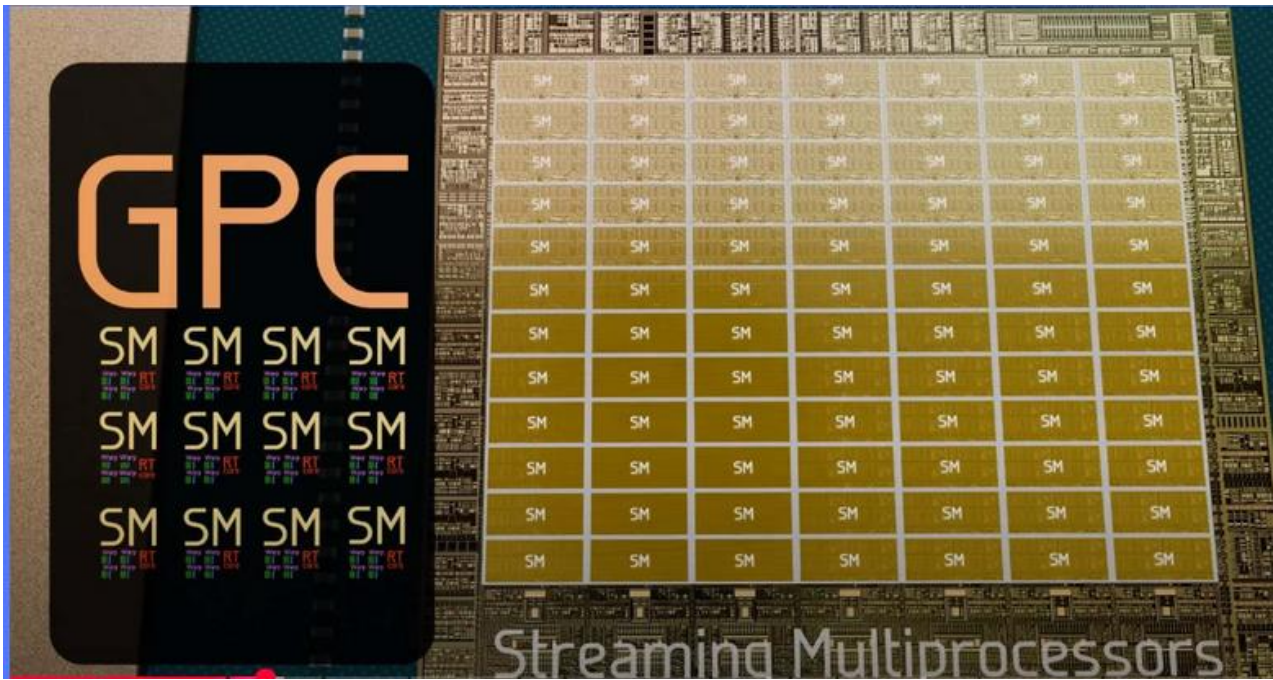
- **Manual Inference Engine:** Developed a C++ system for inference to bypass high-level framework overhead.
- **Tensor Structure:** Designed a custom 3D Tensor class in C++ to manage Channel, Height, and Width (CHW) data layouts.
- **Weight Extraction:** Automated the process of extracting pretrained weights from ONNX models into raw binary format for C++ loading.
- **Validation:** Verified implementation accuracy by comparing C++ output logits with PyTorch reference results on 50,000 images.



```
Images: 50000
Top 1 accuracy: 0.084
Top 5 accuracy: 0.59
Total time: 287.024 sec
Throughput: 174.201 images per sec
Average latency per image: 3.51641 ms
```


GPU Architecture & Scaling Benchmarks

- **Hardware Profiling:** Studied the hierarchy of GPU compute units, including Graphics Processing Clusters (GPCs) and Streaming Multiprocessors (SMs).
- **Warp Execution:** Analyzed the impact of warp divergence on kernel performance, specifically for conditional branching in convolutions.
- **Batch Size Scaling:** Measured the performance of MobileNet across batch sizes ranging from 16 to 1024 to find the GPU saturation point.
- **Metric Analysis:** Evaluated average image time (ms) vs. throughput (img/sec) to identify latency-throughput trade-offs.



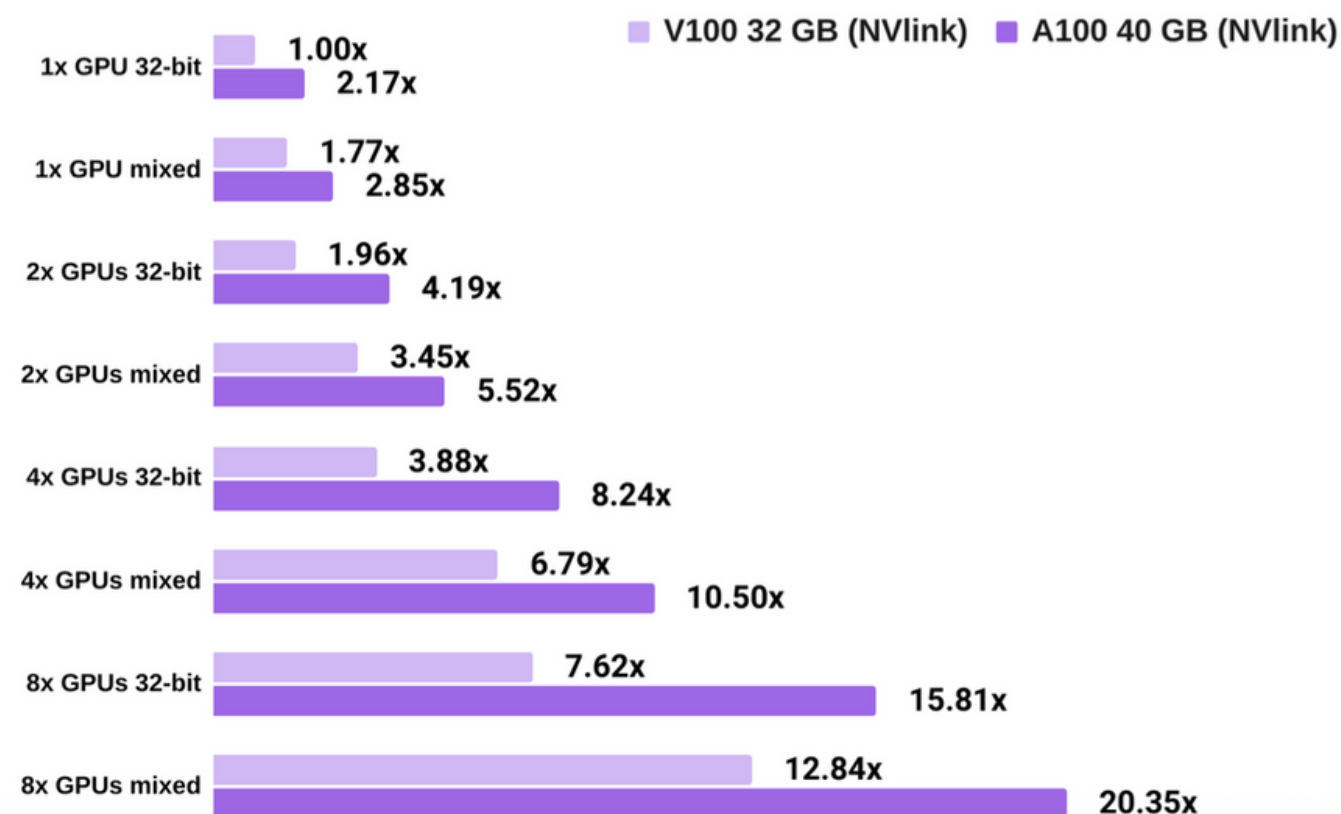
=== Batch Size GPU Benchmark Results ===								
Batch Size	Total Images	Total Time (s)	Avg Batch Time (s)	Avg Image Time (ms)	Throughput (img/sec)	Top-1 Acc (%)	Top-5 Acc (%)	
16	50000	303.55	0.0971	6.07	164.7	69.06	88.52	
32	50000	160.89	0.1029	3.22	310.8	69.06	88.52	
64	50000	96.20	0.1230	1.92	519.7	69.06	88.52	
128	50000	59.68	0.1526	1.19	837.8	69.06	88.52	
256	50000	46.04	0.2349	0.92	1086.0	69.06	88.52	

=== Batch Size Benchmark Results ===								
Batch Size	Total Images	Total Time (s)	Avg Batch Time (s)	Avg Image Time (ms)	Throughput (img/sec)	Top-1 Acc (%)	Top-5 Acc (%)	
16	50000	1239.64	0.3967	24.79	40.3	69.06	88.52	
32	50000	888.11	0.5682	17.76	56.3	69.06	88.52	
64	50000	886.90	1.1341	17.74	56.4	69.06	88.52	
128	50000	838.35	2.1441	16.77	59.6	69.06	88.52	
256	50000	1211.01	6.1786	24.22	41.3	69.06	88.52	
512	50000	1010.06	10.3067	20.20	49.5	69.06	88.52	

Cloud Computing Performance (A100 vs. V100)

- **High-End GPU Comparison:** Benchmarked inference performance on NVIDIA A100 (Google Colab) vs. V100 (Akya CUDA) architectures.
- **Mixed Precision:** Investigated the throughput gains provided by Tensor Cores when operating in FP16 precision.
- **Inference Frameworks:** Compared standard PyTorch execution against optimized ONNX Runtime and TensorRT workflows on Ampere architecture.

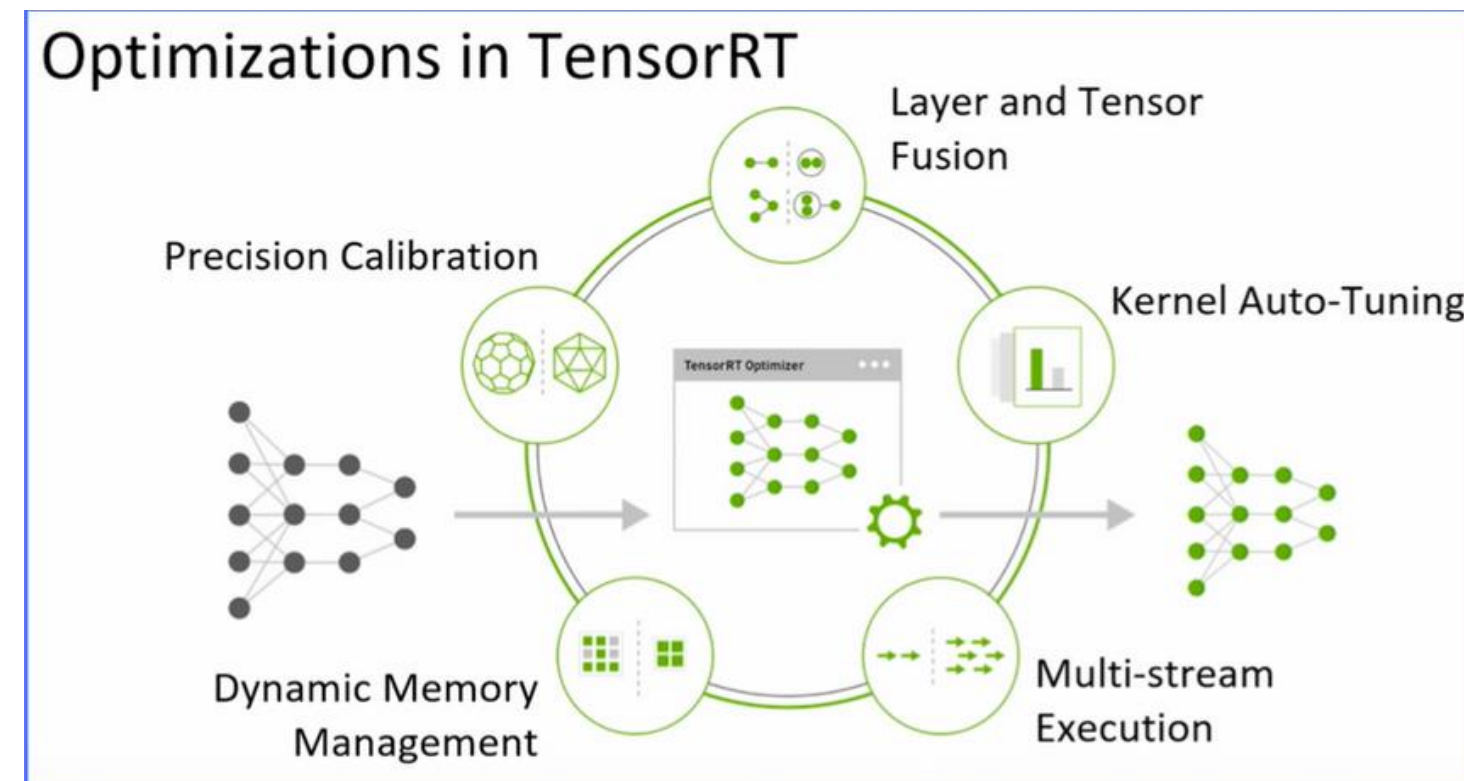
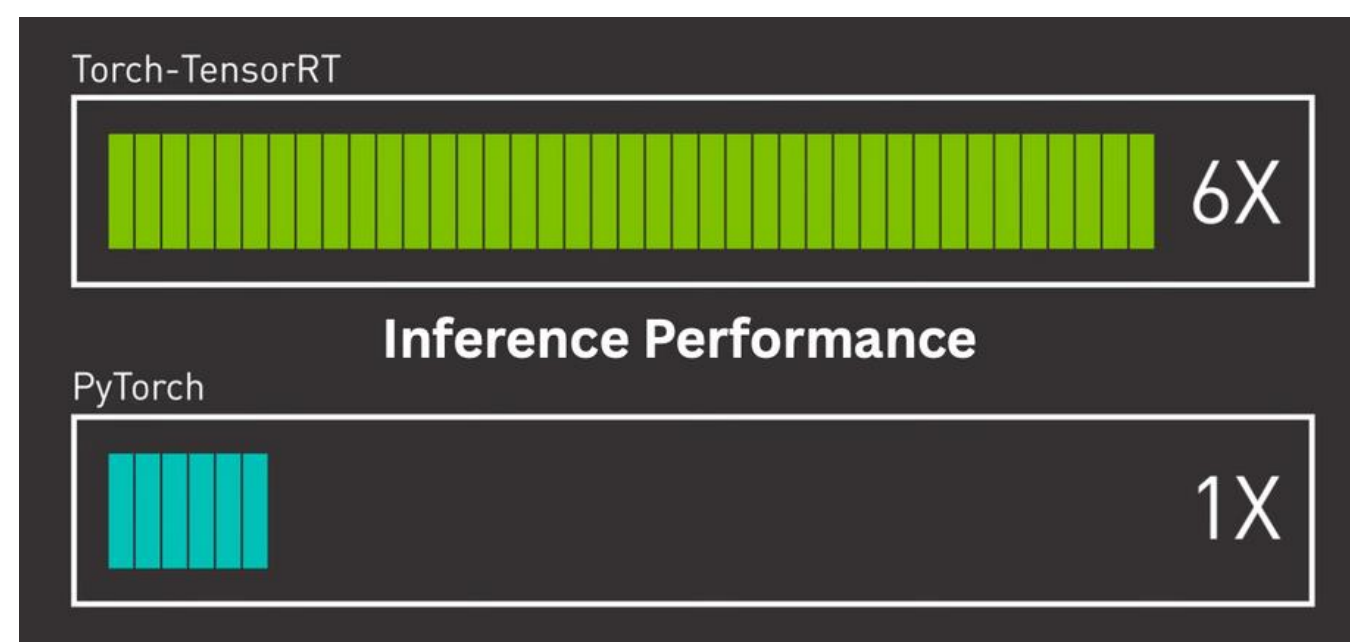
A100 vs V100 convnet training speed, PyTorch



```
PyTorch Latency: 27.914714336395264 ms
PyTorch FPS: 35.82340080393364
ONNX Runtime Latency: 2.971470355987549 ms
ONNX Runtime FPS: 336.5337291637414
TensorRT FP16 Latency: 0.38864803314208984 ms
TensorRT FP16 FPS: 2573.022155587237
```


TensorRT Optimization Workflow

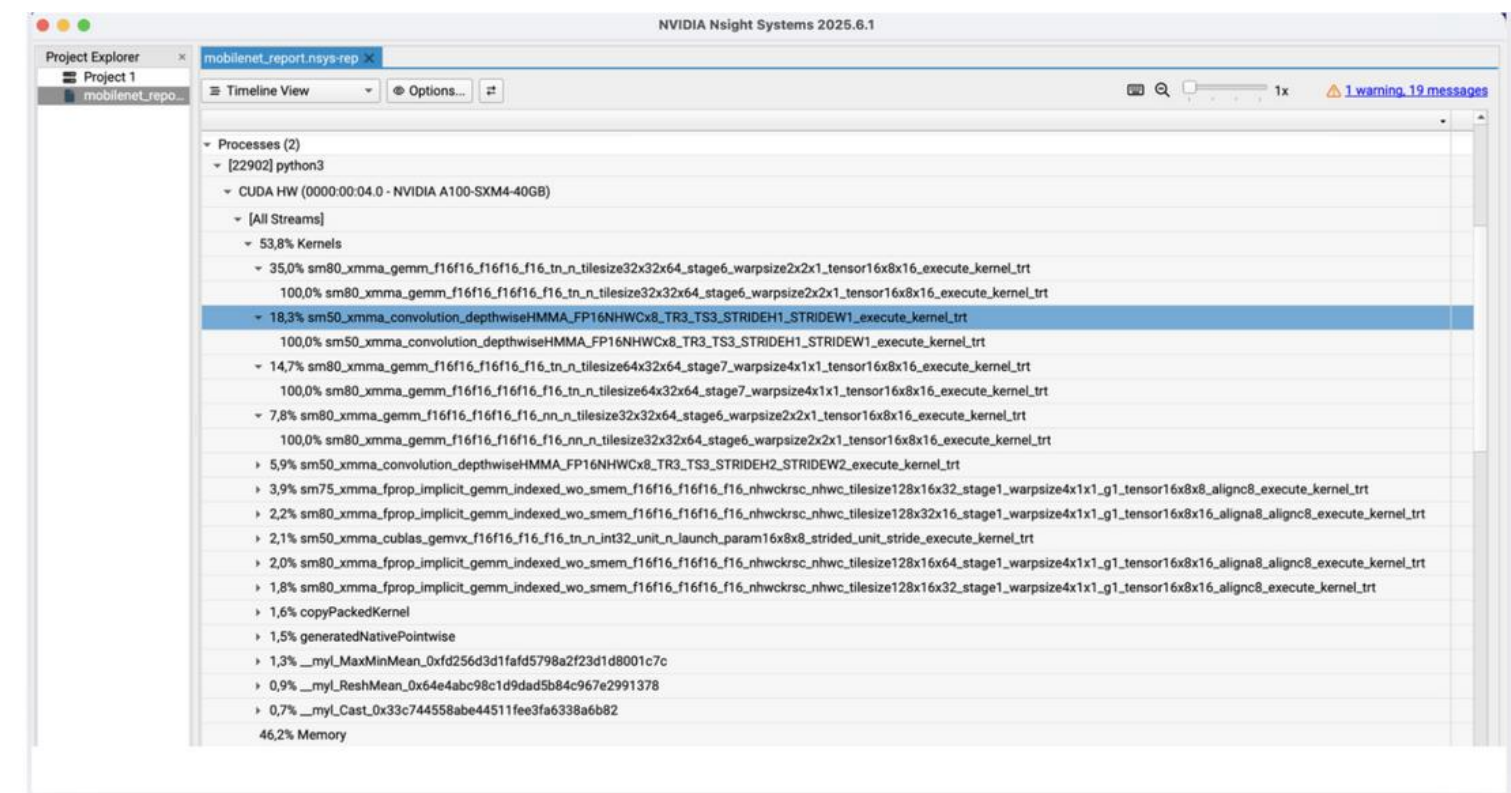
- **Optimization Pipeline:** Established a conversion path from PyTorch → ONNX → TensorRT Engine for production-grade inference.
- **Layer Fusion:** Utilized TensorRT's vertical and horizontal layer fusion to reduce memory bandwidth requirements.
- **Kernel Auto-tuning:** Allowed the engine to select the best CUDA kernels for specific GPU architectures (e.g., SM80 for A100).
- **Performance Gains:** Achieved significant throughput increases over standard GPU inference.



Custom CUDA Kernel Design & Profiling

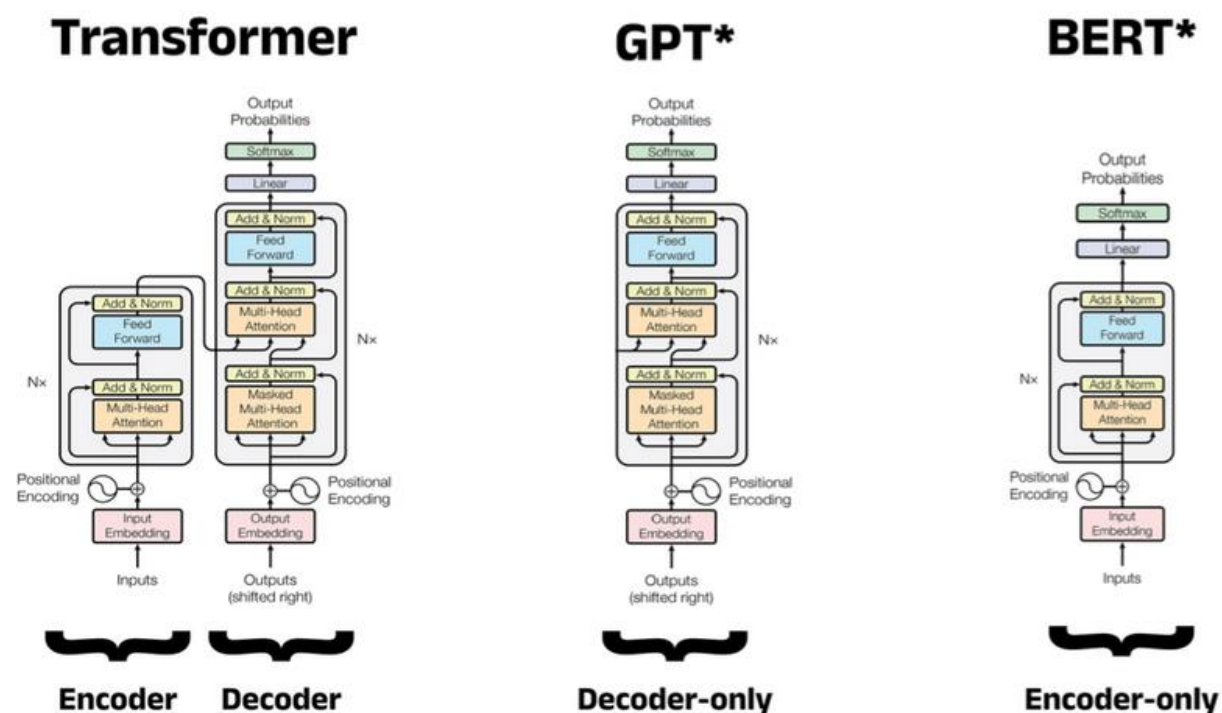
- **Custom Depthwise Kernel:** Developed a specialized FP16 CUDA kernel for 3x3 depthwise convolutions to outperform generic PyTorch kernels.
- **Design Choices:** Simplified kernels to support fixed NCHW layouts, removing unnecessary branching to improve thread utilization.
- **Profiling with Nsight Compute:** Analyzed compute-bound vs. memory-bound bottlenecks using NVIDIA Nsight tools.
- **Kernel Fusion:** Investigated the benefits of fusing activation functions (ReLU6) directly into the convolution kernel.

```
PyTorch FP16 depthwise conv: 0.018766 ms  
  
Custom CUDA kernel: 0.009842 ms  
  
Speedup (PyTorch / Custom): 1.91x
```



Large Language Model (BERT) Optimization

- **Cross-Domain Application:** Applied MobileNet optimization principles to Large Language Models (LLMs) like BERT.
- **Sequence Length Impact:** Measured the effect of padding and sequence length (SEQ_LEN) on inference latency.
- **Backend Comparison:** Tested PyTorch FP16 vs. ONNX Runtime vs. TensorRT for NLP tasks.
- **Throughput Scaling:** Demonstrated how TensorRT provides extreme speedups (~11x) for transformer-based architectures.



BERT ON A100

PyTorch FP16 latency: 8.156 ms | Throughput: 122.6 seq/s
ONNX Runtime CUDA latency: 4.123 ms | Throughput: 242.5 seq/s
TensorRT FP16 latency: 0.700 ms | Throughput: 1428.7 seq/s

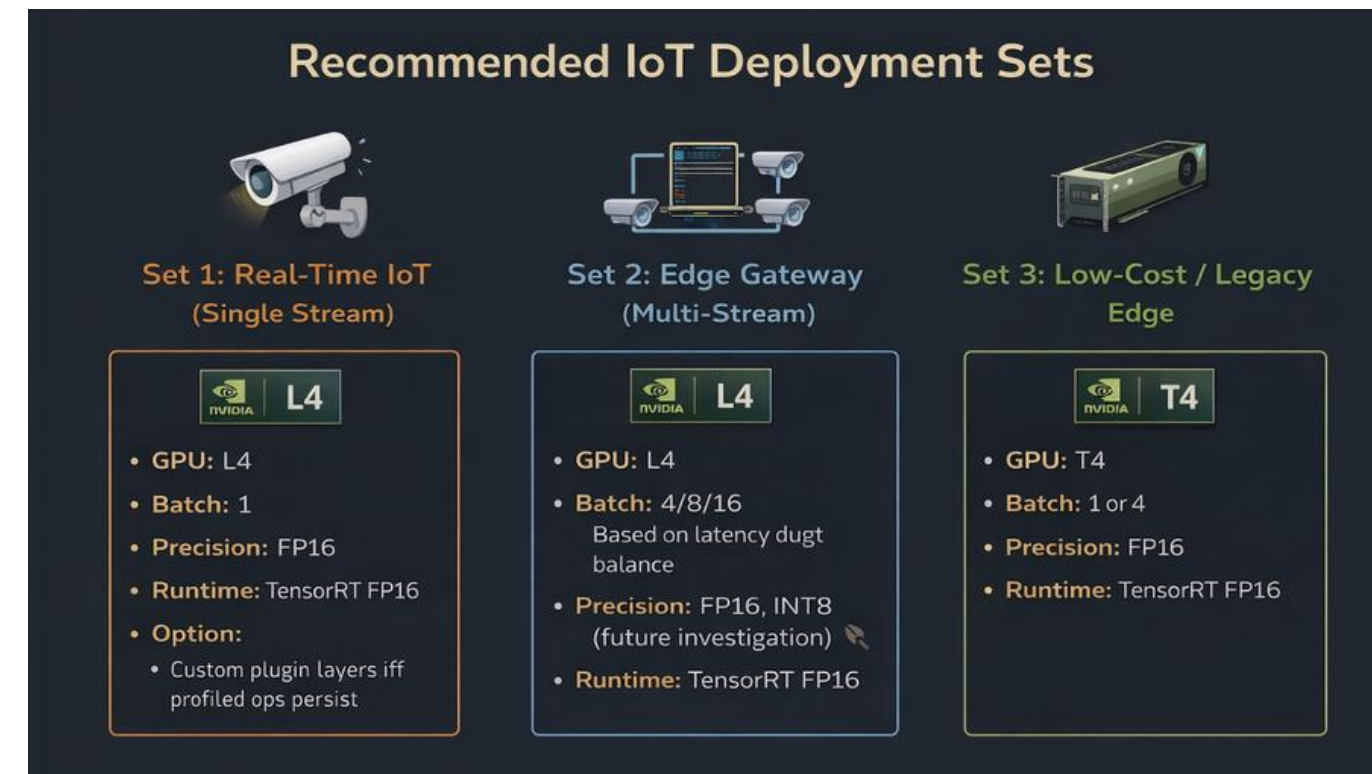
MOBILENET ON A100

PyTorch Latency: 22.01592445373535 ms
PyTorch FPS: 45.42166748897679
ONNX Runtime Latency: 2.53363037109375 ms
ONNX Runtime FPS: 394.6905639469056
TensorRT FP16 Latency: 0.34925031661987305 ms
TensorRT FP16 FPS: 2863.275858067176

Performance Standards for IoT Deployment

- **IoT Latency Targets:** Researched real-time vision requirements, targeting <33ms (30 FPS) for end-to-end processing.
- **Watt-per-Performance:** Evaluated the NVIDIA L4 GPU for edge scenarios where power efficiency is as critical as throughput.
- **Precision Recommendations:** Concluded that FP16 with TensorRT + Custom Kernels is the optimal configuration for modern IoT gateways.
- **Use-Case Scenarios:** Defined deployment sets for real-time single-stream vs. multi-stream edge gateways.

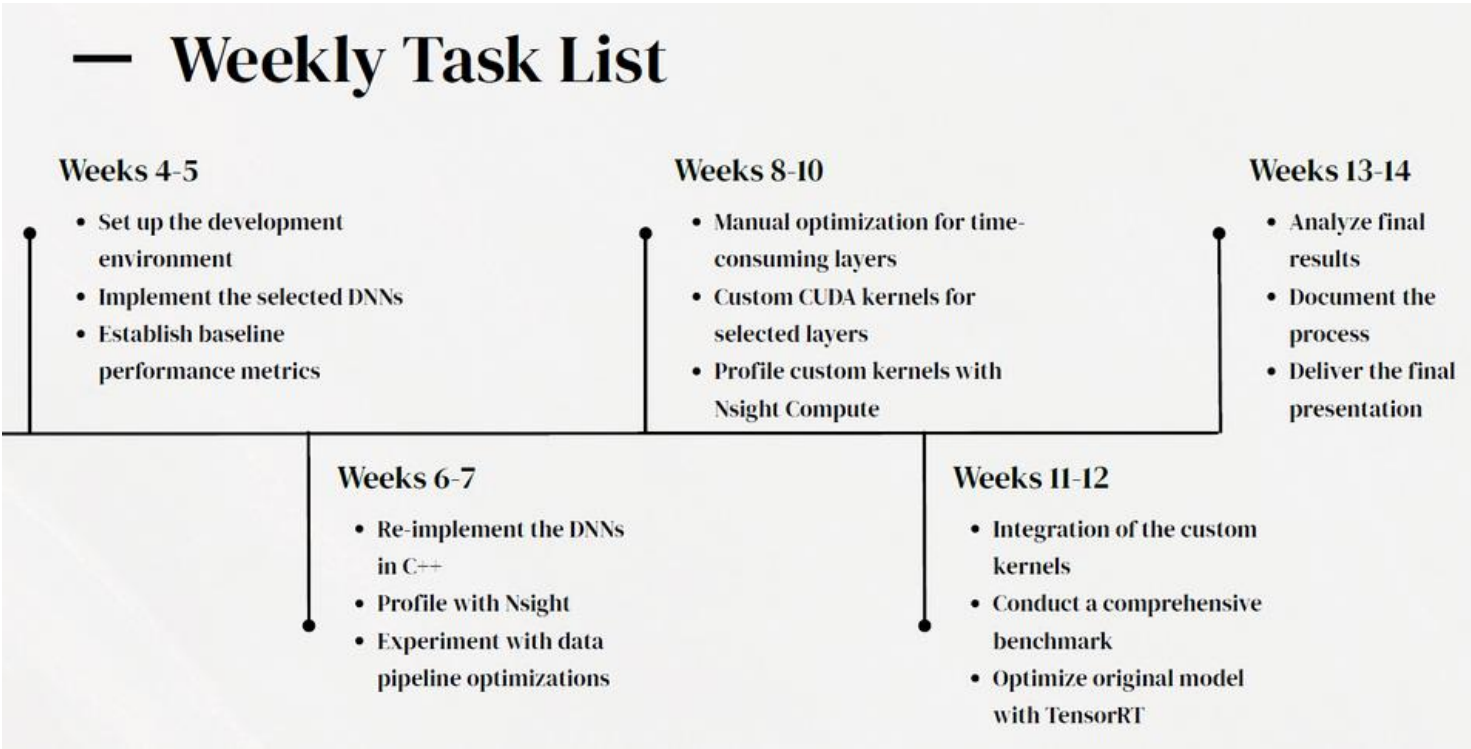
Bileşen	Seçim	Gerekçe
GPU	NVIDIA L4	A100 daha hızlı olsa da, L4 özellikle edge veya inference yoğun iş yükleri için tasarlanmıştır ve watt başına yüksek performans sunar.
Batch Size	1	IoT senaryolarında yanıt süresini, yani latency'yi en aza indirmek için kritiktir. Sonuçlarımıza göre, L4 ve A100 üzerinde Batch 1 için 0.23 ms ile 0.34 ms arasında latency göstermektedir.
Precision	FP16	Test edilen tüm GPU'larda latency ve throughput açısından FP32'ye kıyasla tutarlı biçimde daha iyi performans göstermektedir, hızlanma bazı durumlarda 2 kata kadar çıkmaktadır.
Engine	TensorRT + Custom Kernel Plugin	Özel depthwise kerneli 2.07 kat, pointwise kerneli ise 2.61 kat hızlanma sağlayarak standart PyTorch FP16 uygulamasını belirgin şekilde geride bırakmaktadır.



ENS 492 Implementation: EfficientNet

Project Foundations & Literature Review

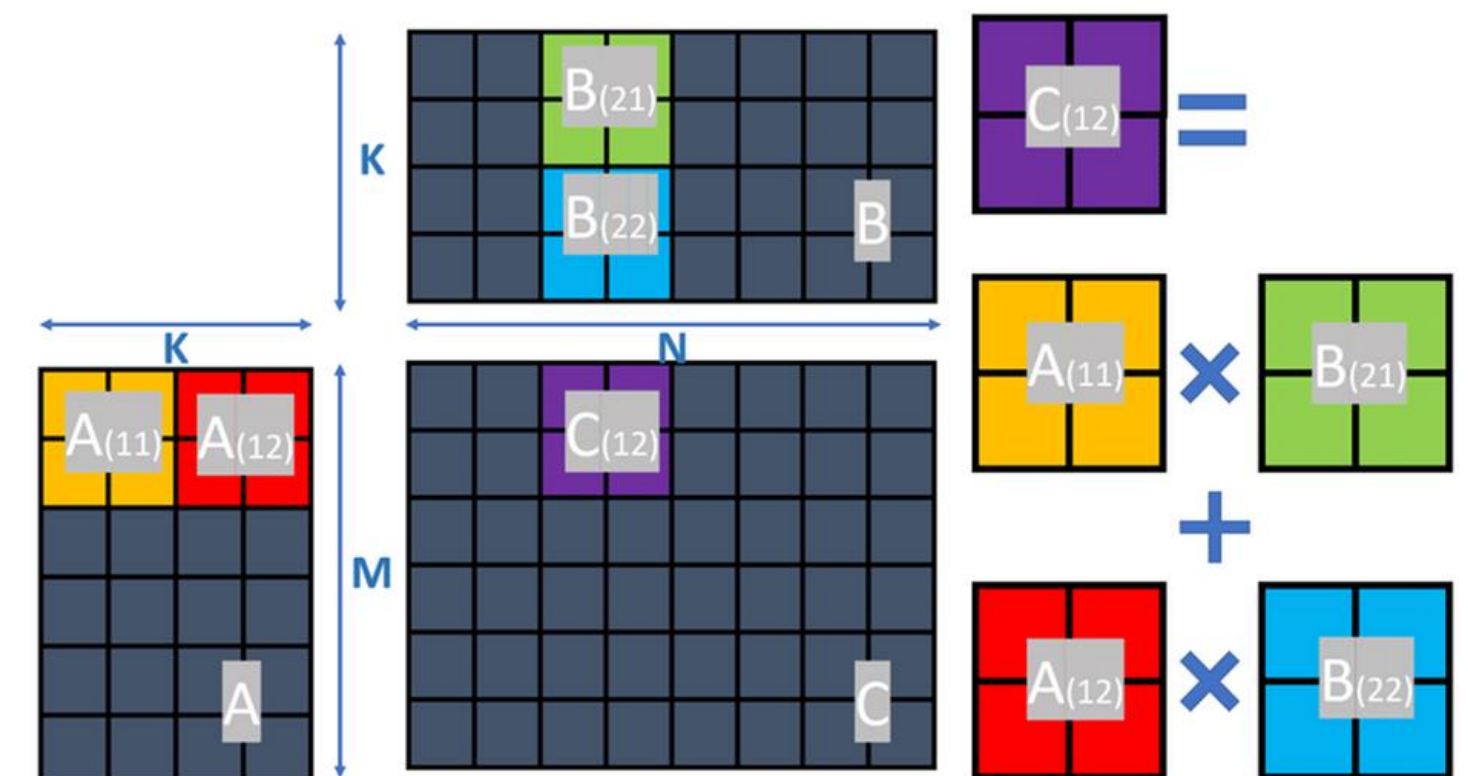
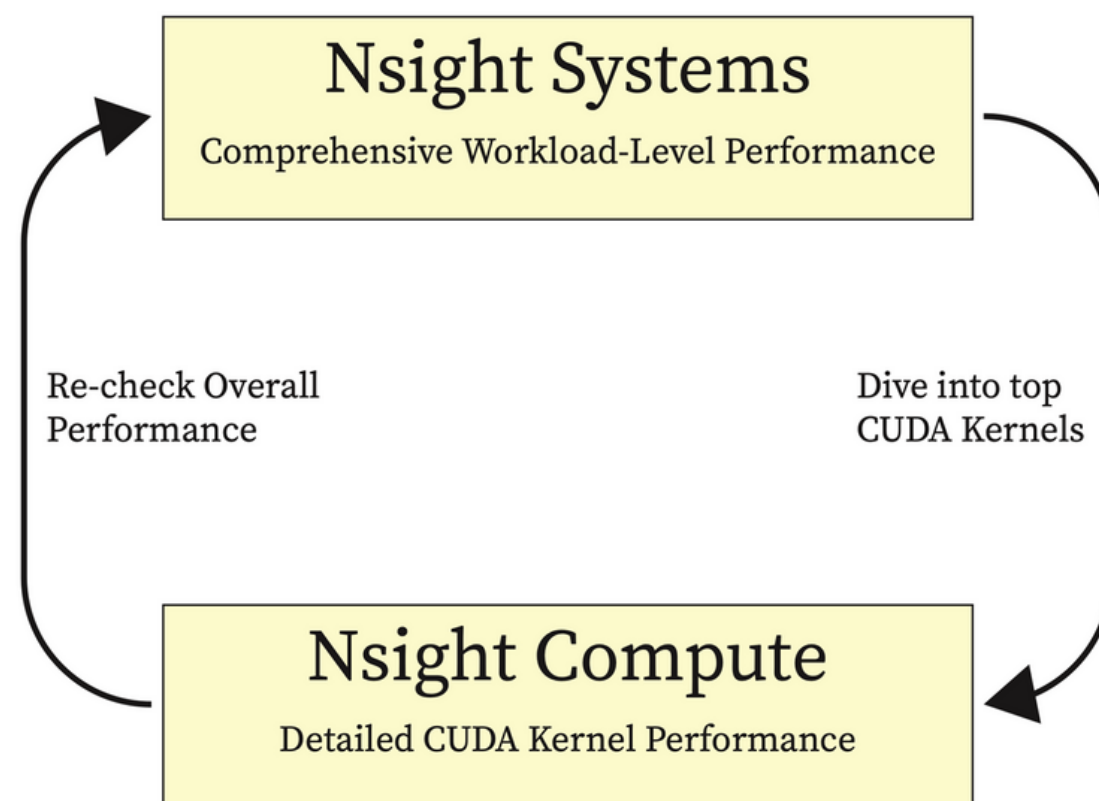
- **Project Objectives:** Defined the core goal of optimizing deep learning models for high-performance inference on GPU-based systems.
- **Literature Analysis:** Conducted a deep dive into existing research regarding GPU-accelerated inference and various neural network architectures.
- **Methodology Design:** Established a systematic pipeline involving model selection, performance profiling, and iterative optimization steps.
- **Environment Setup:** Configured the initial development platform on TRUBA to ensure access to necessary CUDA and Python libraries.



Optimization Area	Technique	Impact	Project Relevance
Data Pipeline	Multi-threaded Data Loading	Overcomes CPU bottlenecks in data augmentation/loading.	Use num_workers > 1 in PyTorch DataLoader when feeding image batches.
Host-Device Transfer	Pinned Memory	Enables faster, asynchronous cudaMemcpyAsync.	Set pin_memory=True in the DataLoader to accelerate CPU-to-GPU data copies.
Host-Device Transfer	Batch Size Tuning	Amortizes transfer and kernel launch overhead.	Experiment with the largest batch size that fits in the VRAM to maximize throughput.
Kernel Execution	Coalesced Memory Access	Maximizes global memory bandwidth.	Ensure adjacent threads in a warp access adjacent memory locations when reading model weights or feature maps.

Framework Selection & Profiling Tools

- **Library Selection:** Evaluated PyTorch, ONNX and TensorRT as the primary frameworks for building and deploying the inference pipeline.
- **Profiling Toolset:** Integrated NVIDIA Nsight Systems and Nsight Compute to identify hardware bottlenecks and kernel execution times.
- **Initial Benchmarks:** Performed baseline testing on standard convolutional models to establish a starting point for future speedup comparisons.
- **Hardware Evaluation:** Assessed the compute capabilities of various NVIDIA GPUs available in the TRUBA cluster for heterogeneous testing.



EfficientNet-B0 Benchmarking and Batch Optimization

- **System Configuration:** Established a baseline using PyTorch and CUDA 12.0 on a Tesla V100 GPU within a SLURM-managed environment.
- **Performance Scaling:** Tested batch sizes from 8 to 1024, achieving peak throughput of 3046.2 img/s before encountering memory limits.
- **Efficiency Analysis:** Identified batch 16 as the most efficient setting at 353.1 img/ms for optimal edge/IoT performance.
- **Strategic Recommendations:** Advised batch 16–32 for low-latency edge tasks and 64–128 for high-throughput server inference.

Batch size	64
Iterations	100
Throughput (<u>img/s</u>)	2824.3
Avg Latency (<u>ms/batch</u>)	22.7
Memory Allocated (MB)	65.5
Memory Reserved (MB)	1002.0
<u>nvidia-smi</u> util (%)	100
<u>nvidia-smi</u> mem_ used (MB)	1390
<u>nvidia-smi</u> mem_ total (MB)	16384

Batch	Throughput (<u>img/s</u>)	Latency (<u>ms/batch</u>)	Efficiency (<u>img/ms</u>)
16	2376.3	6.73	353.1
8	1463.2	5.47	267.5
32	2643.6	12.10	218.5
64	2824.8	22.66	124.7
128	2936.7	43.59	67.4
256	3000.7	85.31	35.2
512	3031.5	168.89	17.9
1024	3046.2	336.15	9.1

C++ Inference Transition and Environment Setup

- **Inference Transition:** Aimed to move from Python-based training to high-performance C++ inference using EfficientNet-B0 to maximize Images Per Second (IPS) on NVIDIA GPUs.
- **Optimization Pipeline:** Established a four-step workflow involving model export to ONNX, engine building with TensorRT, and creating a C++ runtime for optimized memory management.
- **Technical Implementation:** Successfully exported the model using the PyTorch timm library and compiled C++ binaries by manually resolving missing development headers and linker dependencies.
- **Week's Progress:** Validated GPU hardware access and code logic despite a runtime failure caused by library version mismatches between Conda and Pip environments on the cluster.

EfficientNet-B0 Optimization and C++ Benchmarking

- **Weekly Goal:** Re-implement and optimize EfficientNet-B0 for high-performance inference, focusing on throughput and latency.
- **Benchmark Strategy:** Prioritized latency-critical edge applications by fixing the batch size at 1, where CPU overhead typically dominates execution time.
- **Environmental Resolution:** Resolved deep-seated library conflicts by deploying an Apptainer container to align OS, CUDA, and TensorRT versions.
- **Optimization Outcomes:** Achieved 1,023 FPS and 0.97 ms latency using pinned memory and reusable buffers, extracting 84% of hardware's limit.

```
&&&& RUNNING TensorRT.trtexec [TensorRT v8603] # trtexec
...
[12/01/2025-15:16:05] [I] === Performance summary ===
[12/01/2025-15:16:05] [I] Throughput: 1218.89 qps
[12/01/2025-15:16:05] [I] Latency: min = 0.856201 ms, r
[12/01/2025-15:16:05] [I] Enqueue Time: min = 0.448608
[12/01/2025-15:16:05] [I] H2D Latency: min = 0.0556641
[12/01/2025-15:16:05] [I] GPU Compute Time: min = 0.794
[12/01/2025-15:16:05] [I] D2H Latency: min = 0.00512695
...
&&&& PASSED TensorRT.trtexec [TensorRT v8603]
```

🚀 COMPLETE BENCHMARK SWEEP (00 vs 03)

Strategy	Compiler Flag	Latency (ms)	Throughput (FPS)
Baseline (Synchronous)	-00 (Default)	1.18375	844.775
Baseline (Synchronous)	-03	1.09512	913.139
Opt 1: Reusable Buffers	-00 (Default)	1.08961	917.761
Opt 1: Reusable Buffers	-03	1.09986	909.209
Opt 2: Async (Pageable)	-00 (Default)	1.08299	923.366
Opt 2: Async (Pageable)	-03	1.08761	919.448
Opt 3: Async (Pinned)	-00 (Default)	0.976615	1023.94
Opt 3: Async (Pinned)	-03	1.04207	959.631
Opt 4: Double Buffer	-00 (Default)		
Opt 4: Double Buffer	-03		

Kernel Profiling and Custom CUDA Development

- **Layer Optimization:** Shifted from timeline profiling to detailed kernel analysis using Nsight Compute to perform manual layer tuning.
- **Bottleneck Identification:** Discovered that Depthwise Separable Convolutions and Swish activations are the primary performance limiters.
- **Performance Speedup:** Achieved a 1.85x speedup per layer, reducing latency by 45% with custom kernels (0.0047 ms) versus native TensorRT (0.0087 ms).
- **Technical Progress:** Developed custom CUDA kernels in C++ to replace suboptimal layers and improve memory bandwidth.

```
, { "name" : "PWN(PWN(/blocks/blocks.0/blocks.0.0/se/gate/Sigmoid), /blocks/blocks.0/blocks.0.0/se/Mul)", "timeMs" : 7.12205, "averageMs" : 0.00874945,
```

```
Apptainer> ./test_swish_real
--- Benchmarking Custom Swish (EfficientNet Layer Size) ---
Elements: 401408
Total Time (10000 runs): 47.5293 ms
Average Kernel Time: 0.00475293 ms
Effective Bandwidth: 675.639 GB/s
```


Kernel Integration & The “Perfect Run”

- **Integration Challenge:** Overcame the "Black Box" issue where TensorRT defaults to slow, separate layers for Swish instead of fused kernels.
- **Graph Surgery:** Implemented a four-step process to programmatically detect "Sigmoid + Multiply" patterns and implant custom SwishPlugin nodes.
- **Verification Success:** Confirmed active plugin status via trtexec profiling on an NVIDIA Tesla P100, ensuring high-performance execution.
- **Benchmark Results:** Achieved a ~65-67% speedup by reducing swish kernel memory transactions from six to two per pixel, reaching 29,852 FPS.

```
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin  
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin  
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin  
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin  
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin  
[I] [TRT] No importer registered for op: SwishPlugin. Attempting to import as plugin.  
[I] [TRT] Searching for plugin: SwishPlugin, plugin_version: 1, plugin_namespace:  
[I] [TRT] Successfully created plugin: SwishPlugin
```

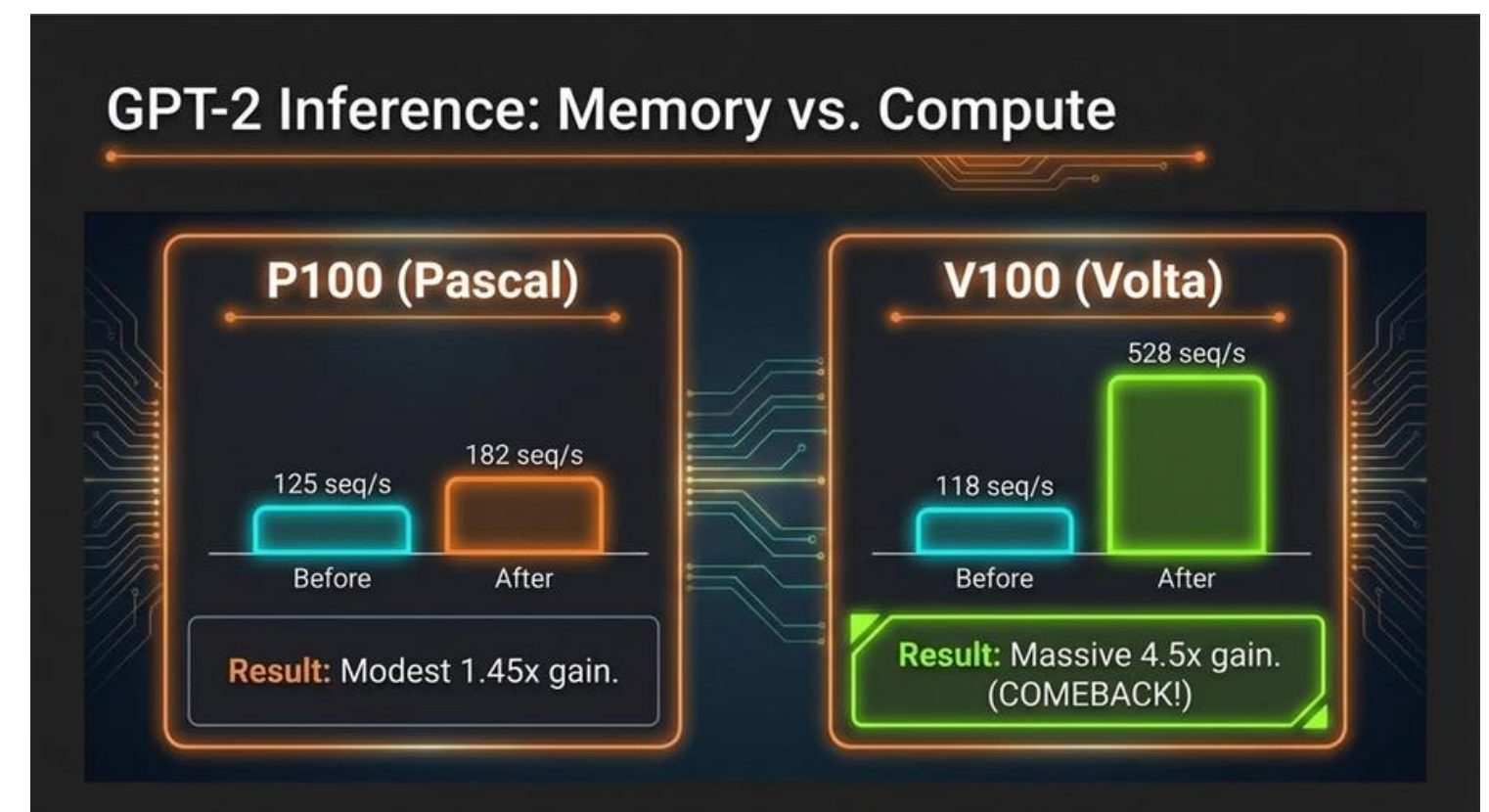
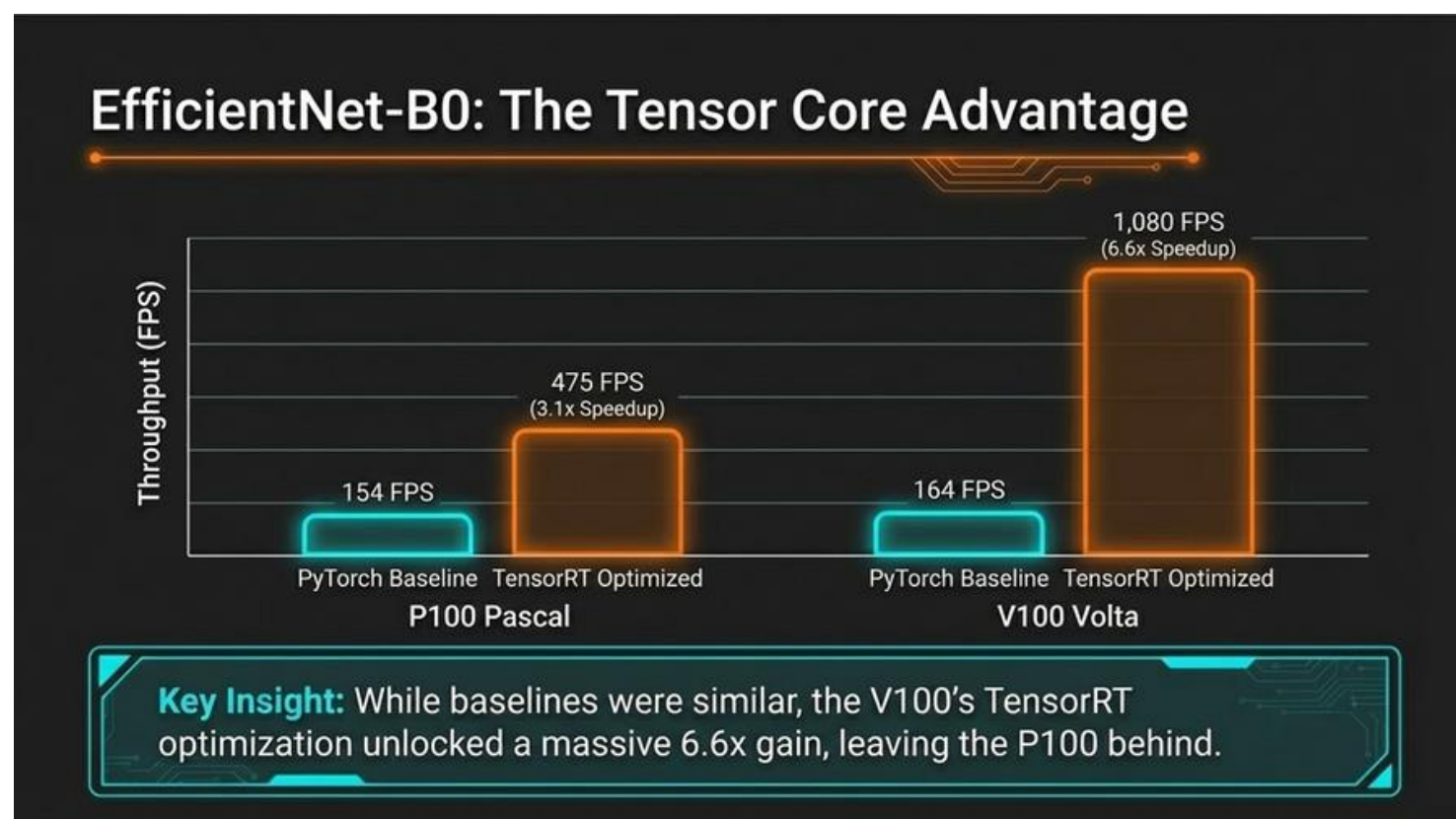
```
[1/3] Building Baseline Engine (No Plugin)...  
-> Baseline FPS (Batch 1): 560.349  
[2/3] Building Custom Engine (With SwishPlugin)...  
->  SUCCESS: SwishPlugin is ACTIVE in the engine!  
[3/3] Running Batch Sweep...
```

Batch	Custom FPS (Yours)	Baseline FPS (NVIDIA)

1	936.635	567.914
16	14877.8	9092.86
32	29852.3	18056.7

Legacy vs. Modern GPU Architecture Analysis

- **Pipeline Standardization:** Developed a universal FP16 benchmarking script using 1,000 iterations to eliminate noise.
- **Vision Optimization:** Reached 1,080 FPS on V100 via Tensor Cores, a 6.6x gain over baseline.
- **Transformer Throughput:** Secured a 4.5x gain for GPT-2 on V100 by shifting to TensorRT (C++).
- **Deployment Insights:** Validated P100 for basic video analytics, while V100 is mandatory for real-time LLMs.



Thank you!